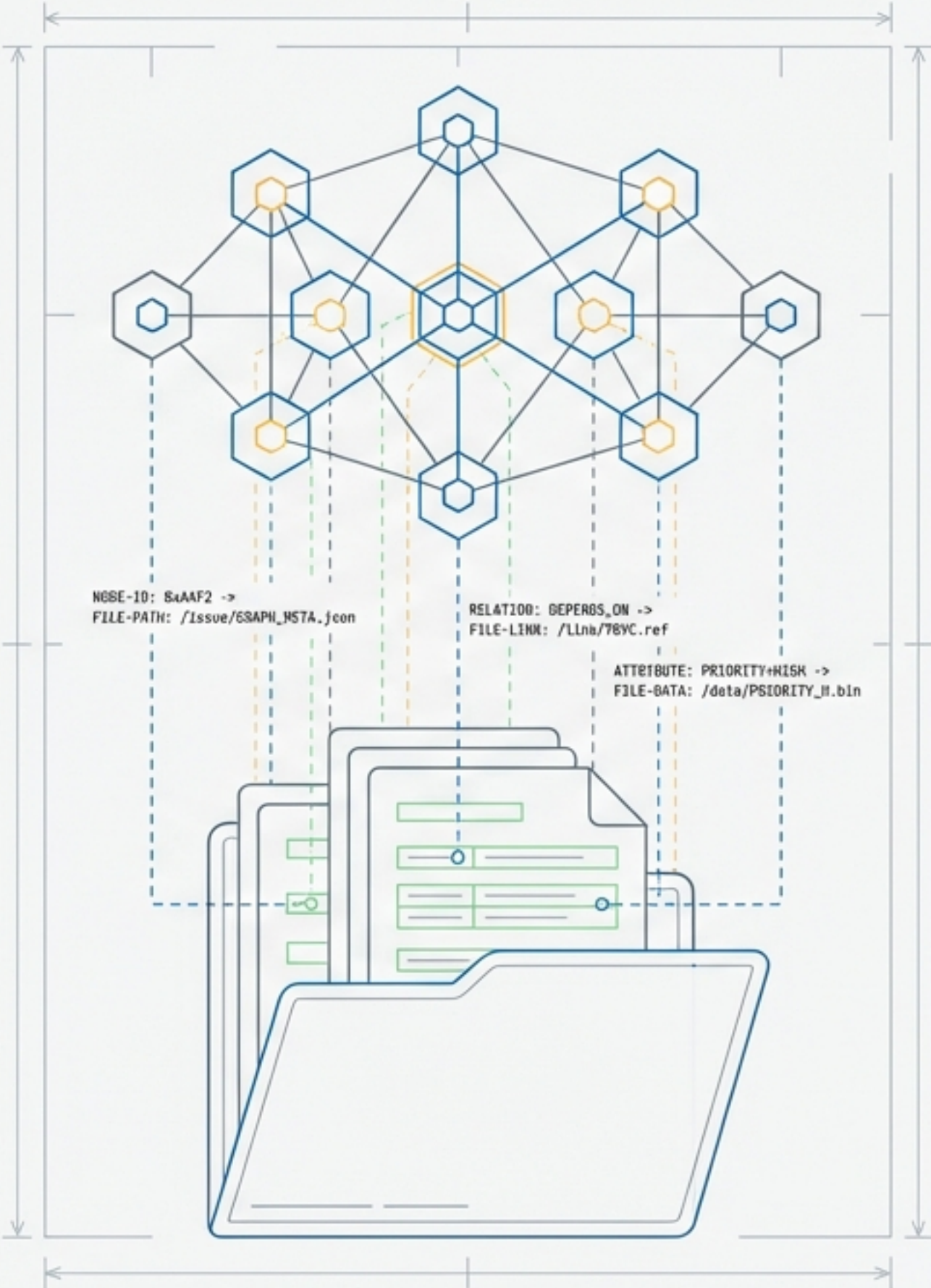


Backend Brief: Memory-FS Integration

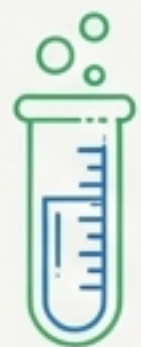
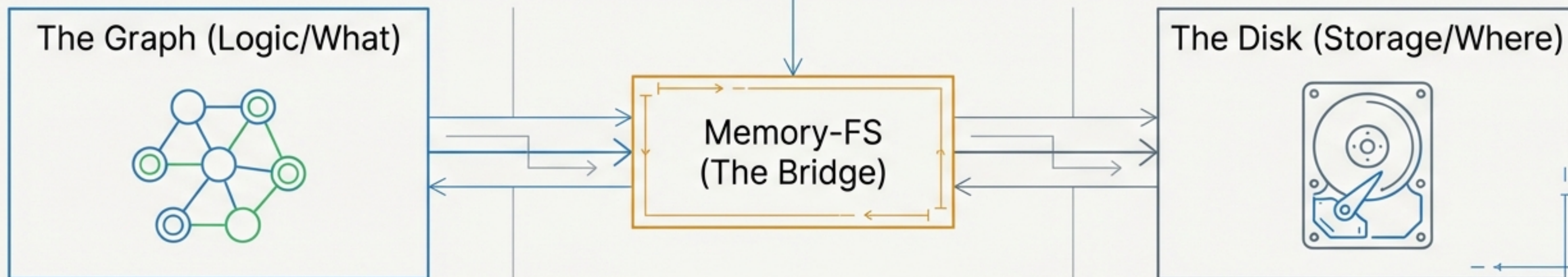
For Graph-Based Issue Tracking System

Version:	0.2.4
Status:	Architecture Design
Companion to:	v0.3.0 Graph-Based Issue Tracking System
Memory-FS Version:	v0.36.2+



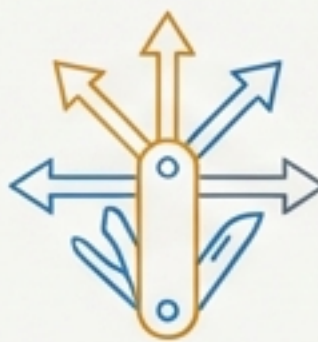
Solving the “Where” so we can focus on the “What”

Memory-FS acts as a storage abstraction layer. It solves the **WHERE problem (storage backend)** allowing the business logic to solve the **WHAT problem (graph nodes and relationships)**.



Testing without I/O

Unit tests run entirely in-memory with zero file system access.



Backend Flexibility

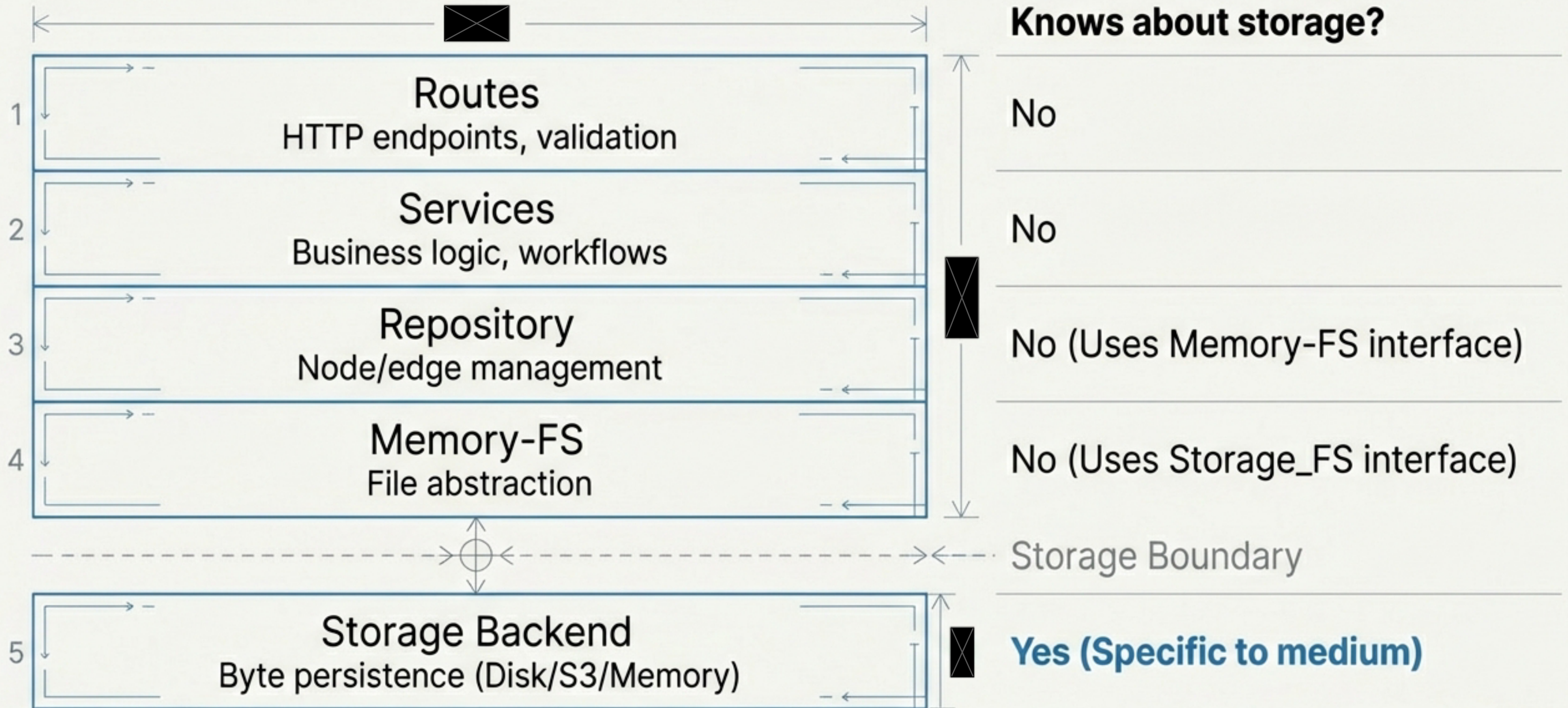
Deploy to local disk (Git), S3 (Cloud), or SQLite (Embedded).



Single Codebase

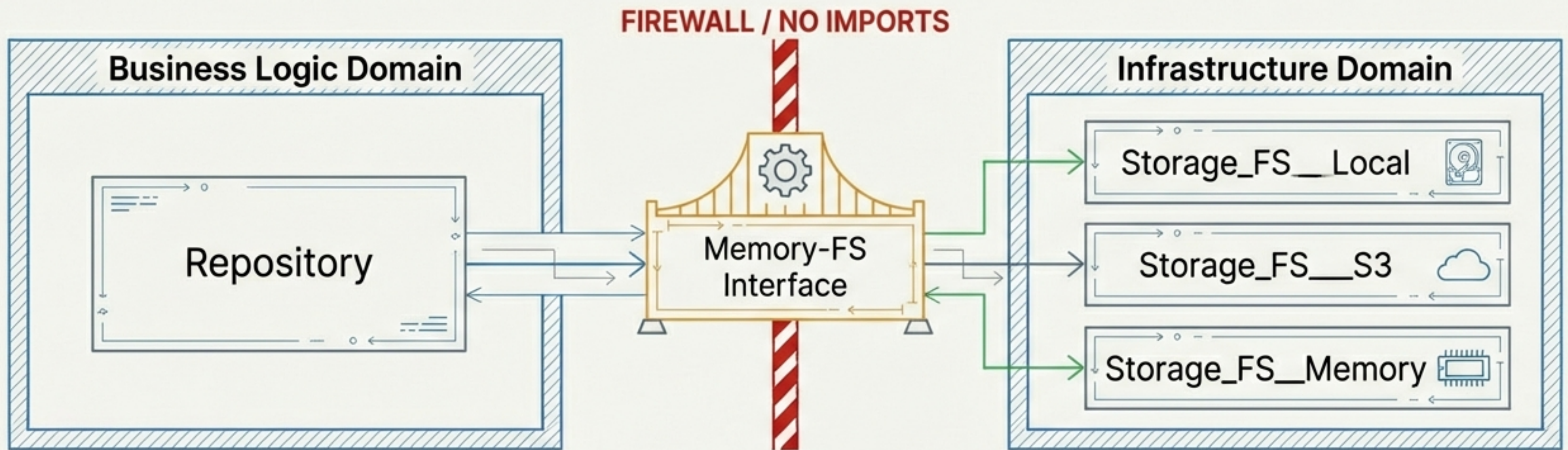
No storage-specific code paths exist within the business logic.

The Architecture Stack



The Golden Rule of Integration

“Our code never imports storage-specific classes.”

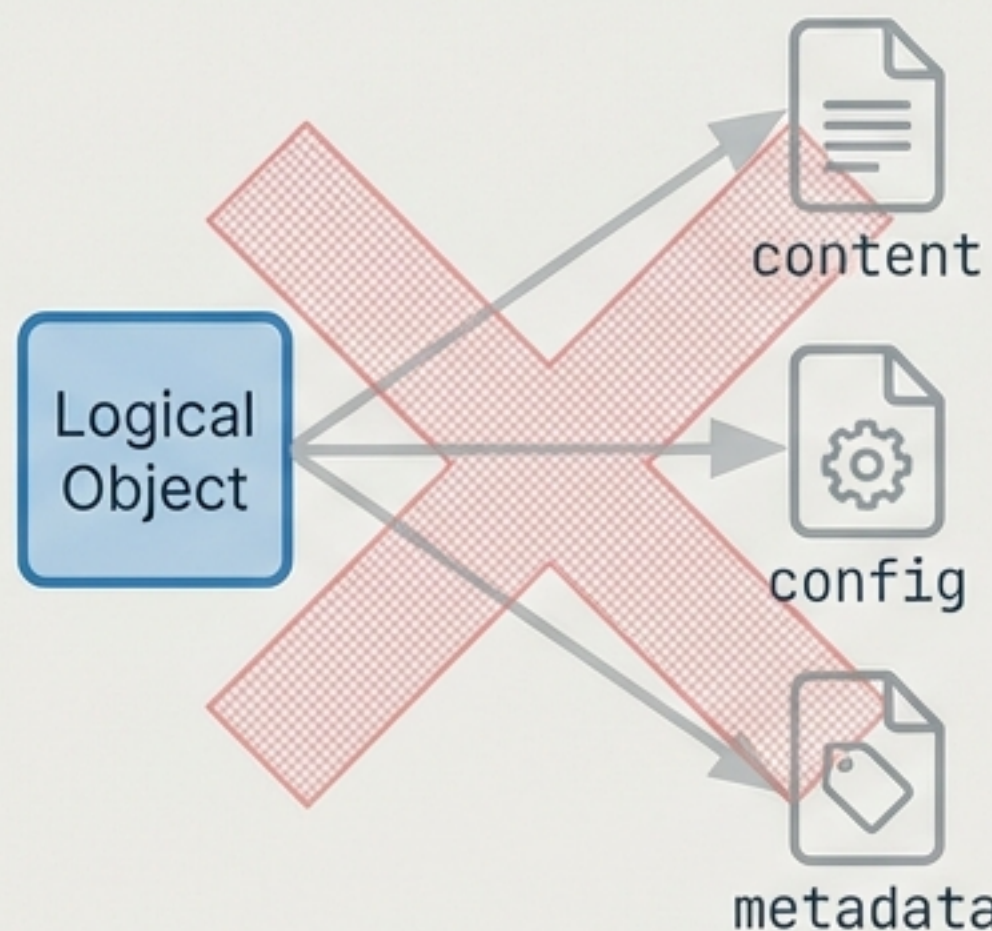


- ① The Repository interacts *only* with Memory-FS. —————> ② Memory-FS interacts *only* with Storage_FS.

Implication: The business logic is hermetically sealed from the filesystem implementation details.

Configuration Strategy: Single-File Mode

Default Mode



Avoids cluttered directories.

Single-File JSON Mode



1 Logical File = 1 Physical File



Git Ready /
Human Readable



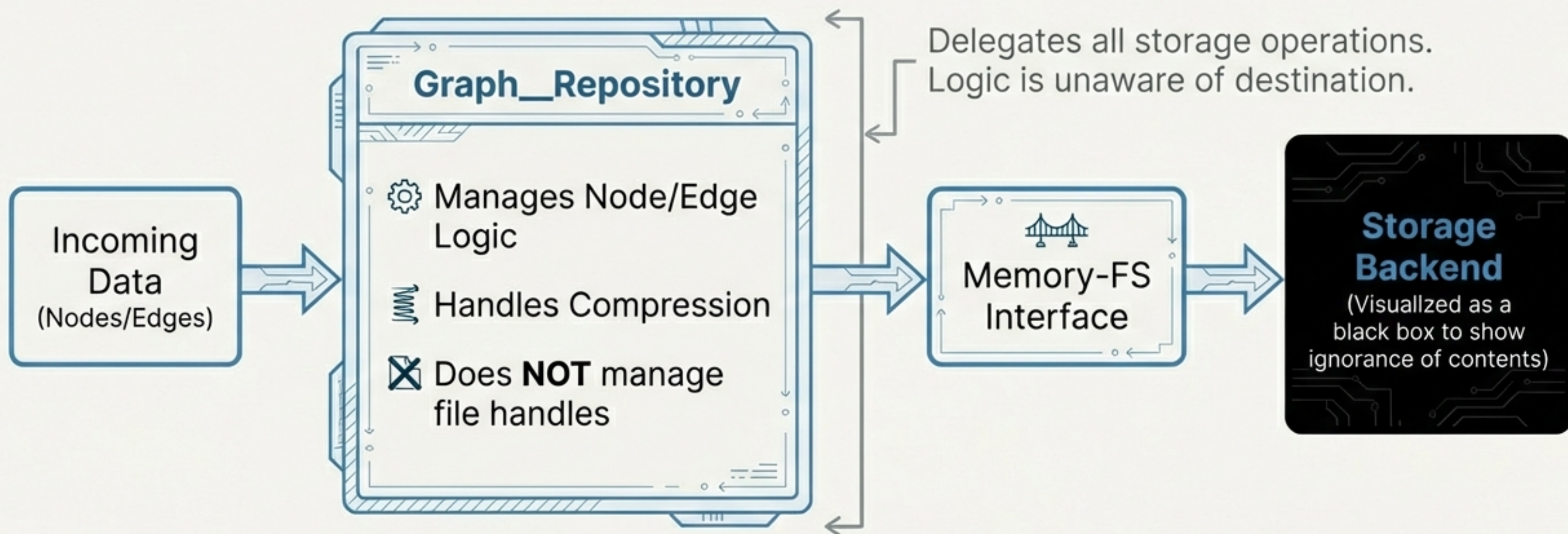
Why: Critical for Git browsability.



Path Handling: A Custom Path Handler generates the graph node directory structure.

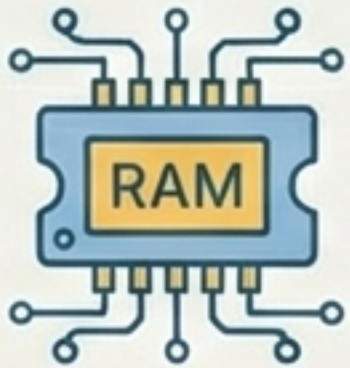
The Repository Implementation

Focus Class: Graph_Repository



Storage Backend Menu: Pick Your Weapon

Storage_FS__Memory



Use Case:
Unit tests, caching

Persistence:
None
(process lifetime)

Storage_FS__Local_Disk



Use Case:
Development,
Git workflows

Persistence:
Files on disk

Storage_FS__Sqlite



Use Case:
Embedded
applications

Persistence:
Single .db file

Storage_FS__S3



Use Case:
Cloud deployment

Persistence:
AWS S3 bucket

Storage_FS__Zip



Use Case:
Archives, export

Persistence:
Single .zip file

Performance & Trade-offs

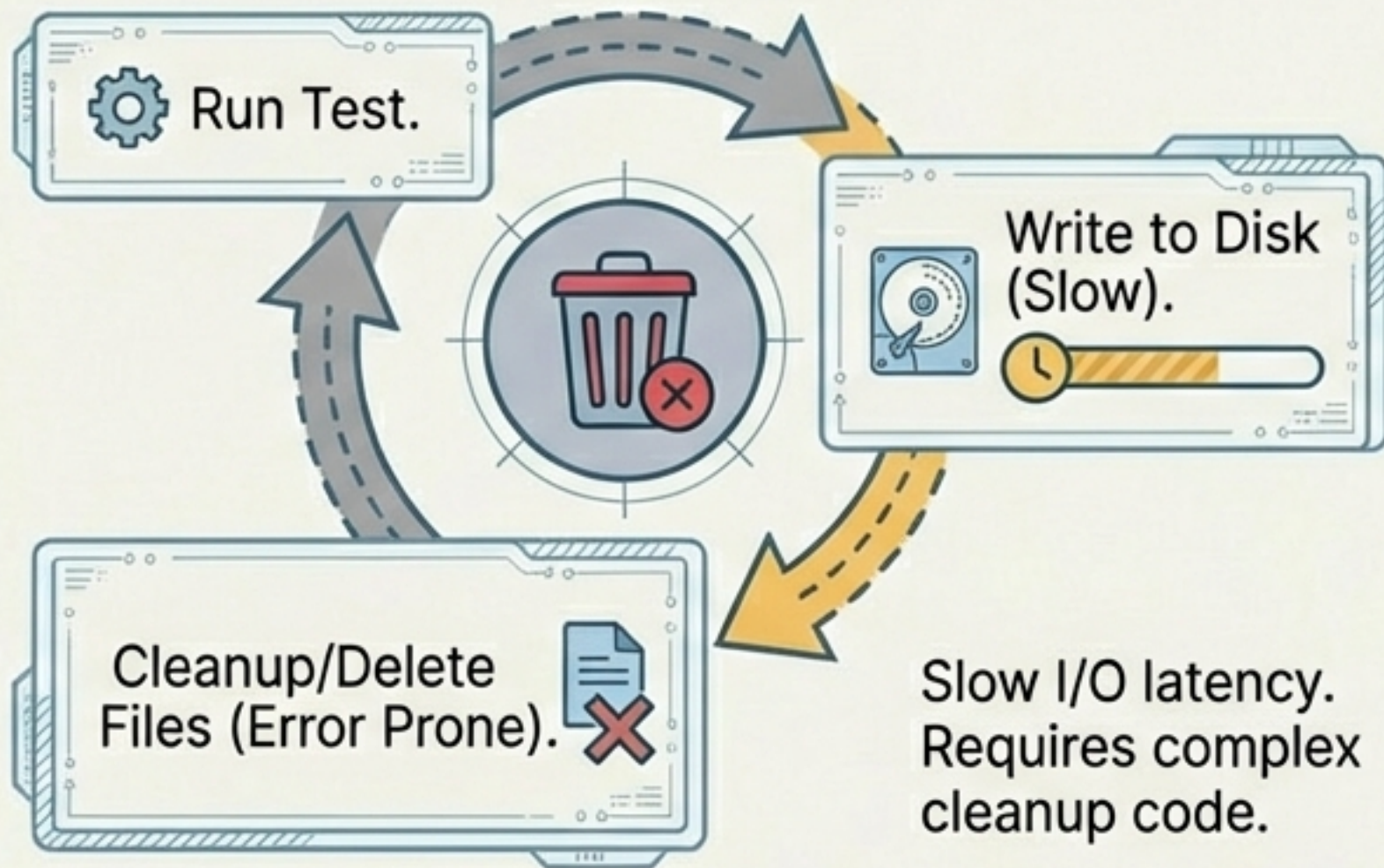
Operation	Memory	Local Disk	S3 (Cloud)
Read	~10 μ s	~1ms	~50-200ms
Write	~10 μ s	~5ms	~100-500ms
Delete	~10 μ s	~1ms	~50-100ms

Listing Complexity:

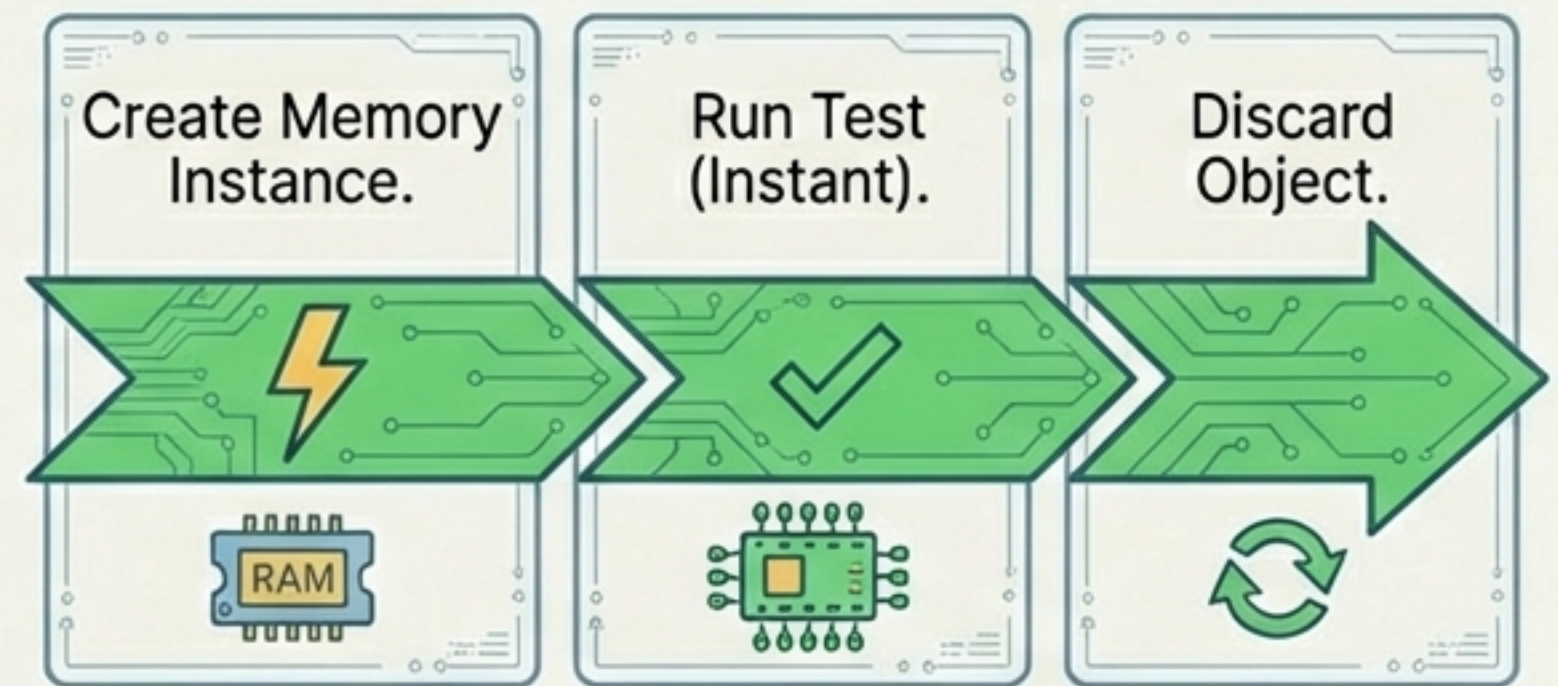
- ⚙ Memory/Local/SQLite: $O(n)$
- 🕒 S3: $O(n)$ + pagination overhead

The Efficiency Gain: In-Memory Testing

The Old Way (File-Based)



The New Standard (In-Memory)



Zero I/O latency.
Zero I/O latency. 100% Isolation.
Fresh slate every time.

Integration Testing: Uses Local Disk backend in a temp directory specifically to verify Git integration.

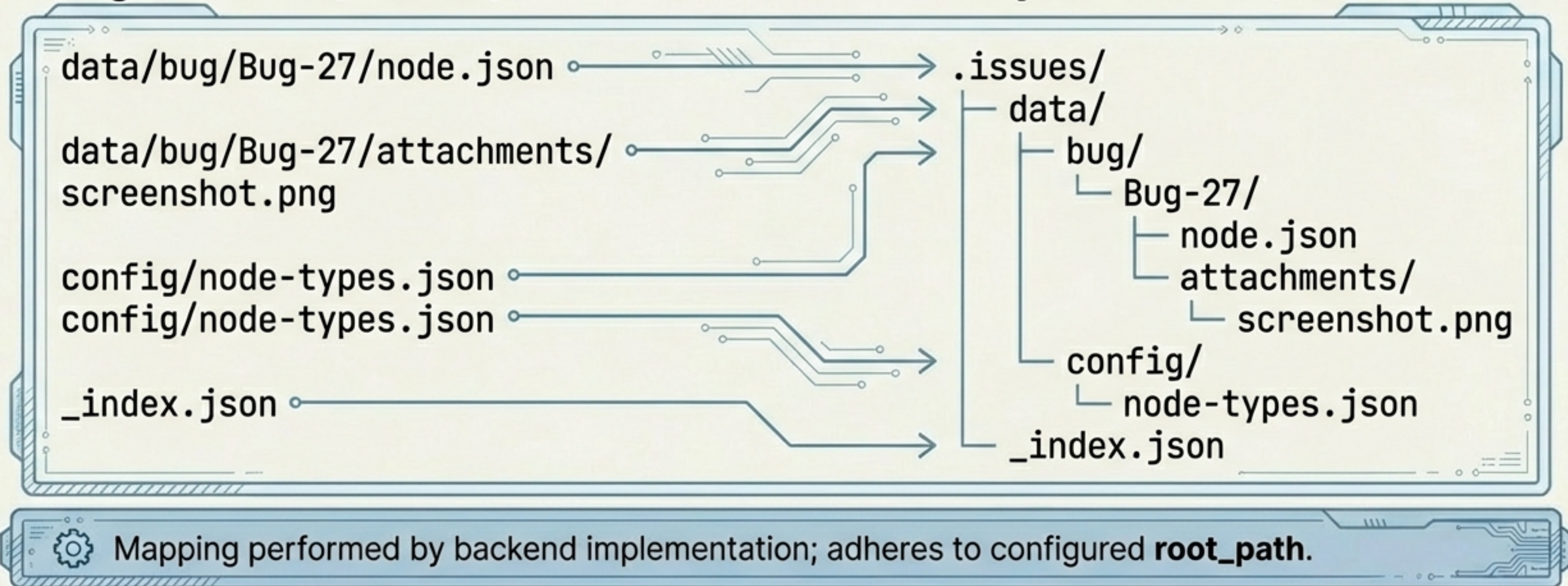
Mapping Reality: Logical to Physical

Context: Local Disk Backend with **root_path**=".issues"

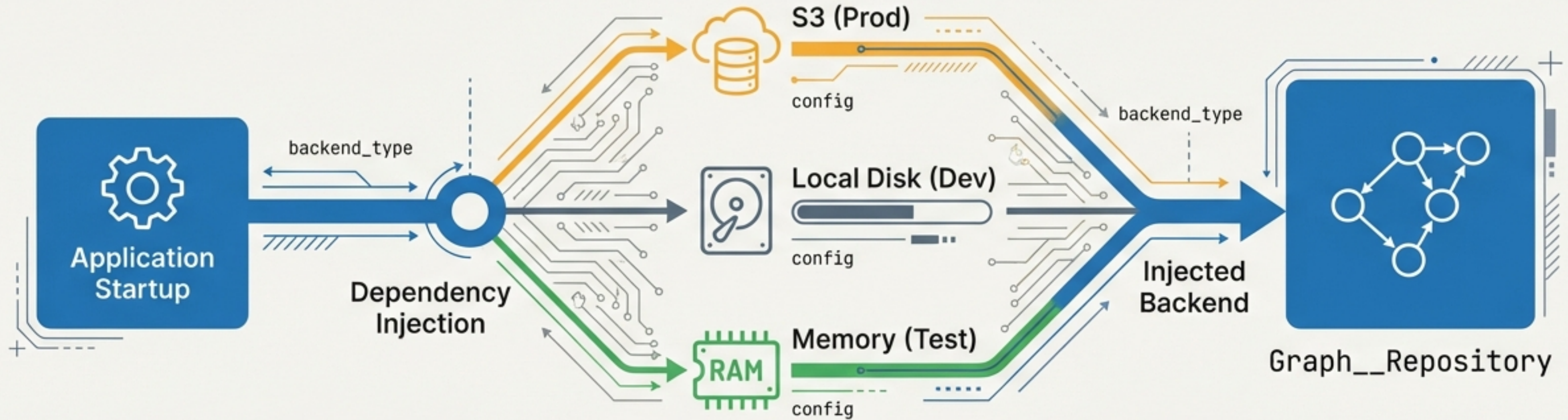


Logical Paths (In Code)

Physical Paths (On Disk)

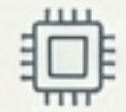


Backend Injection & Selection



1. Repository accepts pre-configured Memory-FS instance.
2. Application selects backend at startup.
3. Code runs identically regardless of selection.

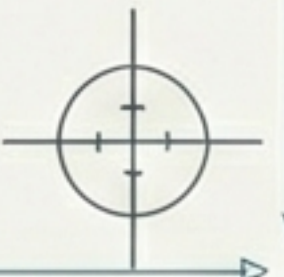
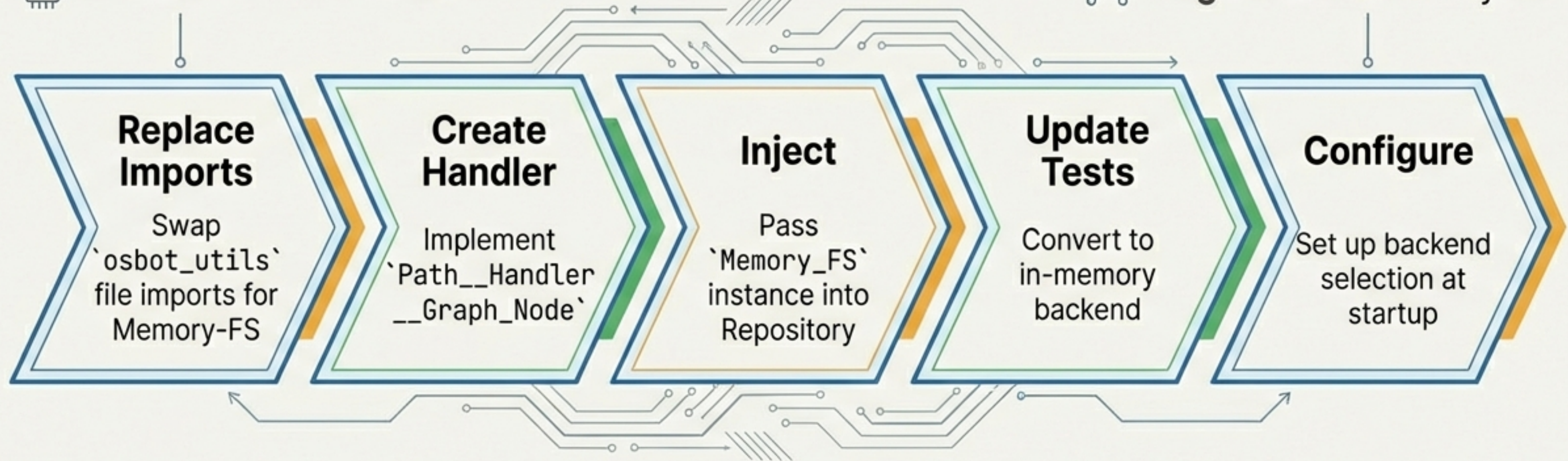
Migration Strategy: From v0.2.2 to v0.2.4



Current State: File-Based



Target State: Memory-FS



Implementation Checklist: Phases 1 & 2

Phase 1: Memory-FS Integration

- ☐ Create Path__Handler__Graph_Node class
- ☐ Create Graph__Repository class with Memory-FS
- ☐ Configure single-file JSON mode
- ☐ Add storage backend factory methods
- ☐ Write unit tests with memory backend

Phase 2: Service Layer

- ☐ Update Node__Service and Link__Service to use Repository
- ☐ Update Type__Service for config files
- ☐ Update Attachment__Service with compression

Implementation Checklist: Phases 3 & 4

Phase 3: Testing

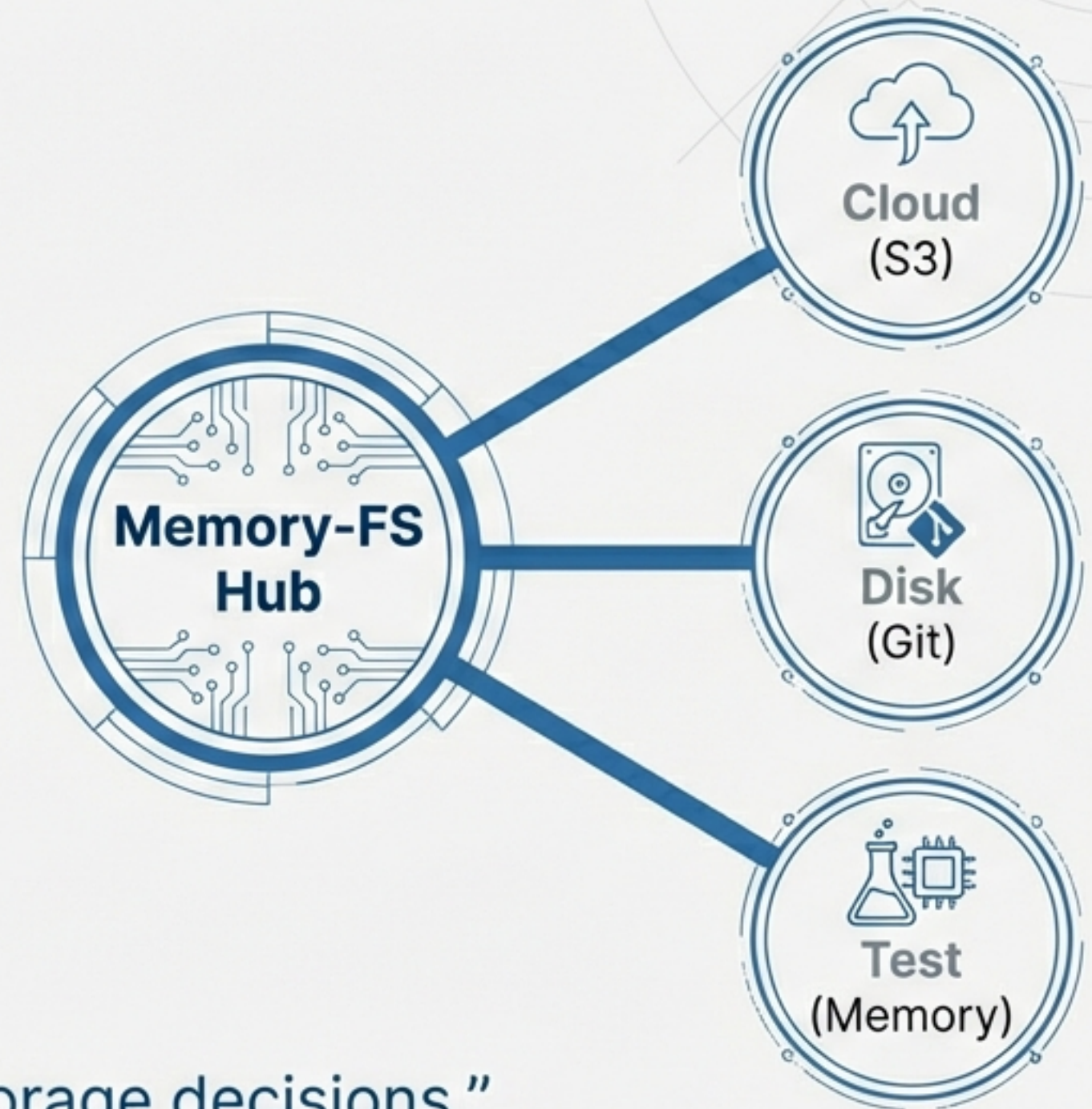
- ☐ Convert all tests to in-memory backend
- ☐ Add integration tests with local disk backend
- ☐ Verify Git browsability of generated files
- ☐ Performance benchmarks for each backend

Phase 4: Deployment Options

- ☐ Document local disk setup for Git workflows
- ☐ Document S3 setup for cloud deployment
- ☐ Document SQLite setup for embedded use
- ☐ Create backend selection configuration

Summary: A Clean Architecture

Concern	Handled By
Node/Edge schemas	Our code (v0.3.0)
Path structure	Path_Handler__Graph_Node
File operations	Memory-FS
Storage medium	Storage_FS backends
Compression	Our code



“Domain logic is completely independent of storage decisions.”